

ES6 Classes — Complete Presentation Guide

Introduction to ES6

What is ES6?

ES6 stands for **ECMAScript 2015**. It is the 6th major version of JavaScript.

ES6 introduced many modern features that made JavaScript:

- Cleaner
- More readable
- Easier to maintain
- More powerful for large applications

One of the most important features introduced in ES6 is:

Classes

1. What are Classes in JavaScript?

A class is a blueprint used to create objects.

Classes allow developers to:

- Reuse code
- Organize data and methods
- Create multiple objects with the same structure

Before ES6, JavaScript used constructor functions and prototypes. ES6 introduced a cleaner and more understandable syntax using classes.

2. Real Life Example of a Class

Think about a:

- Car blueprint
- Student record format
- Mobile phone design

A blueprint is not the real object. It only defines:

- Properties
- Features

- Behavior

Example:

A car blueprint defines:

- Color
- Brand
- Speed
- Start function
- Stop function

Using the blueprint, many cars can be created.

Similarly:

- Class = Blueprint
- Object = Real instance

3. Syntax of ES6 Class

```
class ClassName {  
  constructor() {  
    // initialization code  
  }  
  
  methodName() {  
    // method code  
  }  
}
```

Explanation:

Keyword	Meaning
class	Used to create a class
constructor	Special method executed automatically when object is created
methods	Functions inside class

4. Creating a Simple Class

```
class Student {  
  constructor(name, age) {  
    this.name = name;  
  }  
}
```

```
        this.age = age;
    }

    introduce() {
        console.log(`My name is ${this.name}`);
    }
}
```

Explanation:

constructor()

The constructor initializes object values.

this keyword

`this` refers to the current object.

introduce()

A method inside the class.

5. Creating Objects from Class

Objects are created using the `new` keyword.

```
const student1 = new Student("Ali", 20);
student1.introduce();
```

Output:

```
My name is Ali
```

Explanation:

Code	Meaning
<code>new</code>	Creates new object
<code>student1</code>	Object instance
<code>Student()</code>	Calls constructor

6. Constructor Method

The constructor is a special method.

It:

- Runs automatically
- Initializes properties
- Executes when object is created

Example:

```
class Car {  
  constructor(brand, color) {  
    this.brand = brand;  
    this.color = color;  
  }  
}  
  
const car1 = new Car("Toyota", "Black");  
console.log(car1.brand);
```

Output:

Toyota

Important Points:

- A class can have only ONE constructor.
- Constructor is optional.
- If not written, JavaScript creates a default constructor.

7. Methods in Classes

Methods are functions inside classes.

Example:

```
class Calculator {  
  add(a, b) {  
    return a + b;  
  }  
  
  subtract(a, b) {  
    return a - b;  
  }  
}
```

```
    }  
  }  
  
  const calc = new Calculator();  
  console.log(calc.add(5, 3));
```

Output:

8

8. The this Keyword

`this` refers to the current object.

Example:

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    console.log(`Hello ${this.name}`);  
  }  
}
```

If object name is Ahmed:

Hello Ahmed

Without `this`, JavaScript cannot identify which object's data is being used.

9. Class Expression

Classes can also be stored inside variables.

```
const Employee = class {  
  constructor(name) {  
    this.name = name;  
  }  
};
```

```
const emp1 = new Employee("Usman");
console.log(emp1.name);
```

10. Hoisting in Classes

Classes are NOT hoisted like functions.

Wrong:

```
const obj = new User();

class User {}
```

This causes error.

Correct:

```
class User {}

const obj = new User();
```

11. Static Methods

Static methods belong to the class itself.

They are NOT available to objects.

Example:

```
class MathUtils {
  static multiply(a, b) {
    return a * b;
  }
}

console.log(MathUtils.multiply(2, 3));
```

Output:

Explanation:

- Static methods are called using class name.
 - They are useful for utility functions.
-

12. Getters and Setters

Getters and setters control access to object properties.

Getter

Used to retrieve values.

Setter

Used to update values.

Example:

```
class User {
  constructor(name) {
    this._name = name;
  }

  get name() {
    return this._name;
  }

  set name(value) {
    this._name = value;
  }
}

const user1 = new User("Ali");
console.log(user1.name);

user1.name = "Ahmed";
console.log(user1.name);
```

Output:

Ali
Ahmed

13. Inheritance

Inheritance allows one class to use properties and methods of another class.

Parent class → Base class Child class → Derived class

Syntax:

```
class Child extends Parent {}
```

Example:

```
class Animal {  
  speak() {  
    console.log("Animal makes sound");  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    console.log("Dog barks");  
  }  
}  
  
const dog1 = new Dog();  
dog1.speak();  
dog1.bark();
```

Output:

```
Animal makes sound  
Dog barks
```

Benefits:

- Code reuse
- Better structure
- Easier maintenance

14. super Keyword

`super` is used to access parent class constructor and methods.

Example:

```
class Person {
  constructor(name) {
    this.name = name;
  }
}

class Student extends Person {
  constructor(name, grade) {
    super(name);
    this.grade = grade;
  }
}

const s1 = new Student("Ali", "A+");
console.log(s1.name);
```

Explanation:

- `super(name)` calls parent constructor.
- Must be called before using `this` in child constructor.

15. Method Overriding

Method overriding means child class replaces parent method.

Example:

```
class Animal {
  sound() {
    console.log("Animal sound");
  }
}

class Cat extends Animal {
  sound() {
    console.log("Meow");
  }
}
```

```
const cat1 = new Cat();  
cat1.sound();
```

Output:

```
Meow
```

16. Private Fields

Private fields are hidden from outside access.

They use `#` symbol.

Example:

```
class BankAccount {  
  #balance = 0;  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}  
  
const acc = new BankAccount();  
acc.deposit(1000);  
console.log(acc.getBalance());
```

Benefits:

- Data security
- Encapsulation
- Prevent direct modification

17. Public Fields

Public fields are accessible outside the class.

Example:

```
class Mobile {  
    brand = "Samsung";  
}  
  
const m1 = new Mobile();  
console.log(m1.brand);
```

18. Encapsulation

Encapsulation means:

- Hiding internal details
- Controlling access to data

Using:

- Private fields
- Getters
- Setters

Example:

```
class Employee {  
    #salary;  
  
    constructor(salary) {  
        this.#salary = salary;  
    }  
  
    getSalary() {  
        return this.#salary;  
    }  
}
```

19. Polymorphism

Polymorphism means: One method behaves differently in different classes.

Example:

```
class Shape {  
    draw() {  
        console.log("Drawing shape");  
    }  
}
```

```
    }  
  }  
  
  class Circle extends Shape {  
    draw() {  
      console.log("Drawing circle");  
    }  
  }  
}
```

Same method:

```
draw()
```

Different behavior.

20. Abstraction

Abstraction means: Hiding complex implementation and showing only important features.

JavaScript does not support true abstraction directly like Java or C++.

But abstraction can be achieved using:

- Classes
- Private fields
- Methods

Example:

```
class Car {  
  startEngine() {  
    console.log("Engine Started");  
  }  
}
```

User only sees:

```
startEngine()
```

Internal engine complexity remains hidden.

21. Prototype Behind Classes

ES6 classes internally use prototypes.

Example:

```
class Person {  
  greet() {  
    console.log("Hello");  
  }  
}
```

Internally:

```
Person.prototype.greet
```

Important:

Classes are syntactic sugar over prototype-based inheritance.

22. Difference Between Function Constructor and Class

Function Constructor	ES6 Class
Old syntax	Modern syntax
Less readable	Cleaner
Manual prototype handling	Automatic structure
Difficult for beginners	Easier to understand

Function Constructor Example:

```
function Person(name) {  
  this.name = name;  
}
```

ES6 Class Example:

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

```
}  
}
```

23. Advantages of ES6 Classes

Better Readability

Code looks organized.

Reusability

Classes can create many objects.

Easier Maintenance

Large projects become manageable.

Supports OOP

- Inheritance
- Encapsulation
- Polymorphism

Cleaner Syntax

Less complicated than prototypes.

24. Disadvantages of ES6 Classes

More Memory Usage

Many objects can increase memory.

Learning Curve

OOP concepts may be difficult initially.

Can Be Overused

Simple applications may not need classes.

25. Real World Use Cases

ES6 Classes are heavily used in:

- React Components
- Game Development
- Banking Systems
- Student Management Systems
- Ecommerce Applications
- APIs
- Node.js Applications

Example:

```
class Product {  
  constructor(name, price) {  
    this.name = name;  
    this.price = price;  
  }  
}
```

26. Important Interview Questions

Q1: What is a class?

A blueprint for creating objects.

Q2: What is constructor?

Special method that initializes objects.

Q3: Difference between object and class?

- Class = Blueprint
- Object = Instance

Q4: What is inheritance?

One class acquiring properties and methods of another.

Q5: What is super keyword?

Used to access parent class.

Q6: What is encapsulation?

Hiding data and controlling access.

Q7: What are static methods?

Methods belonging to class itself.

27. Best Practices

Use Meaningful Class Names

Good:

```
class Student {}
```

Bad:

```
class A {}
```

Keep Classes Focused

One class should perform one responsibility.

Use Private Fields for Sensitive Data

```
#password
```

Avoid Large Classes

Break complex logic into smaller classes.

28. Mini Project Example

Bank Account System

```
class BankAccount {
  constructor(owner, balance) {
    this.owner = owner;
    this.balance = balance;
  }

  deposit(amount) {
    this.balance += amount;
  }

  withdraw(amount) {
    this.balance -= amount;
  }

  checkBalance() {
    console.log(`Balance: ${this.balance}`);
  }
}

const user1 = new BankAccount("Ali", 5000);

user1.deposit(1000);
user1.withdraw(500);
user1.checkBalance();
```

29. Summary

ES6 Classes provide:

- Modern syntax
- Better code organization
- Reusability
- OOP support
- Cleaner development structure

Core Topics Covered:

- Class
- Object
- Constructor
- Methods
- this keyword

- Static methods
 - Getters & Setters
 - Inheritance
 - super keyword
 - Encapsulation
 - Polymorphism
 - Abstraction
 - Private fields
 - Prototypes
-

30. Conclusion

ES6 Classes made JavaScript more professional and structured.

Understanding classes is extremely important for:

- React
- Node.js
- Modern JavaScript
- Large-scale web applications

Mastering ES6 classes helps developers write:

- Cleaner code
 - Reusable code
 - Maintainable applications
-

Bonus Practice Tasks

Beginner

1. Create a Student class.
2. Create a Car class.
3. Create a Calculator class.

Intermediate

1. Create Employee and Manager classes using inheritance.
2. Create a Shopping Cart system.
3. Create a Library Management system.

Advanced

1. Build a Banking Application.
2. Build Authentication System.
3. Build Ecommerce Product System.

Final Tip

To master ES6 classes:

- Practice daily
- Build mini projects
- Use inheritance
- Understand prototypes
- Focus on OOP concepts

The more projects you create, the stronger your understanding becomes.